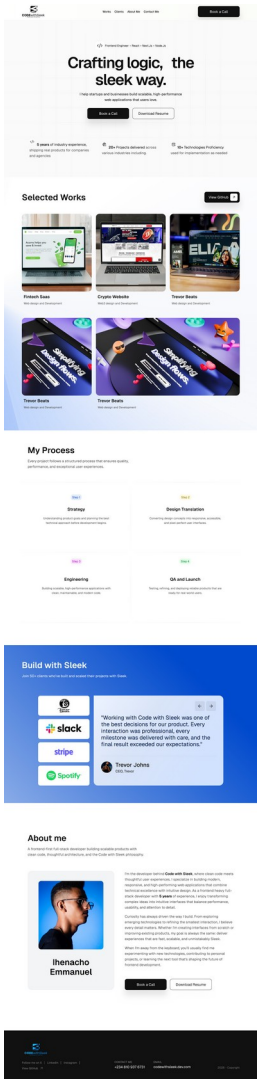
CODE WITH SLEEK / PRODUCT & ENGINEERING BLUEPRINT

Premium Portfolio Platform

Architecture, Content Management, Motion System, Performance Strategy and Sprint Execution Plan



Prepared for Emmanuel Ihenacho - Code with Sleek

Version 1.0 | July 2026

EXECUTIVE SUMMARY

From Portfolio Website to Portfolio Platform

This document defines the complete engineering and product approach for building Code with Sleek as a premium, content-managed portfolio platform. The public experience will combine cinematic but restrained parallax, strong storytelling and project case studies with a secure administration system that allows new work to be published without editing source code.

CORE PRODUCT PRINCIPLE
Content stays server-first and fast. Motion enhances the experience; it never becomes the mechanism through which the content exists.

- **Public experience:** A refined landing page, complete project archive and dynamic case-study pages.
- **Premium motion:** A custom loader, layered page reveal, subtle ambient parallax, interaction-driven depth and selected cinematic sequences.
- **Content operations:** A protected admin dashboard for creating, previewing, publishing, reordering and measuring projects.
- **Verified technical results:** Google PageSpeed/Lighthouse audits stored with dates, device strategy and historical records.
- **Quality gate:** Mobile Lighthouse performance of at least 90, with accessibility, best practices and SEO also above 90 before release.

The intended outcome

The finished product should feel minimal when stationary, layered while scrolling, responsive when touched and cinematic only when the moment deserves it. It must simultaneously demonstrate visual taste, frontend engineering ability, product thinking and performance discipline.

DOCUMENT MAP

Contents

01 / PRODUCT STRATEGY

Product Vision and Scope

The project should be treated as a digital product rather than a static personal website. The design already establishes the public storytelling sequence: hero, credibility metrics, selected work, process, social proof, about and contact. The technical platform must make that experience maintainable and expandable.

Primary objectives

- Position Code with Sleek as a premium frontend-first full-stack engineering brand.
- Demonstrate the ability to combine design translation, motion, architecture and production performance.
- Publish complete project case studies with real evidence, not shallow project cards.
- Make content updates possible through a secure dashboard without source-code changes.
- Protect mobile performance while delivering a distinctive parallax experience.
- Create an architecture that can later support articles, services, analytics and multiple administrators.

Version 1.0 public scope

Area	Included in V1
Landing page	Hero, credibility metrics, selected projects, process, testimonials, about, contact and footer.
Projects archive	All published projects with filters or categories kept lightweight for the first release.
Project details	Structured case-study pages with media, process, challenges, technical architecture, results and performance evidence.
Contact conversion	Book-a-call, email and social pathways with clearly measured CTAs.
Technical SEO	Metadata, sitemap, robots, canonical URLs, social images and structured data.

Version 1.0 admin scope

- Secure administrator login
- Project creation, editing, draft preview and publishing
- Media upload and organisation
- Case-study section builder
- Technology and category management
- PageSpeed audit execution and history

- Testimonials and core site settings
- SEO controls and social image management

Deferred to V1.1 or V2

- Blog or insights publishing
- Multiple roles and approval workflows
- Scheduled publishing
- Advanced project analytics dashboards
- AI-assisted content drafting
- A fully unrestricted visual page builder
- Newsletter and marketing automation

02 / SYSTEM DESIGN

Recommended System Architecture

A modular monolith is the strongest starting point: the public website, protected administration area, API endpoints and data layer live in one Next.js repository, while domains remain separated internally. This provides fast delivery and simple deployment without creating a tightly coupled codebase.

```
Browser
  |-- Public portfolio: /, /projects, /projects/[slug]
  |-- Protected admin: /admin/*
  |
Next.js application
  |-- Server Components and cached queries
  |-- Client motion islands
  |-- Server Actions and Route Handlers
  |-- Authentication and permissions
  |-- SEO and metadata services
  |
Data and external services
  |-- PostgreSQL + Prisma
  |-- Cloudinary or S3-compatible media storage
  |-- Google PageSpeed Insights / Lighthouse data
  |-- Analytics, monitoring and email services
```

Why a modular monolith

- **Lower operational complexity:** One deployment, one repository and one shared type system.
- **Server-first performance:** Public content can be rendered and cached without shipping unnecessary JavaScript.
- **Clear upgrade path:** High-load domains can be extracted later if the product genuinely requires it.
- **Shared validation:** The database schema, admin forms and public renderers use the same TypeScript contracts.
- **Deployment safety:** Content, admin and public routes can be tested together in one CI pipeline.

Rendering strategy

Surface	Rendering and cache approach
Landing page	Primarily static/server-rendered, revalidated after content

	changes; only motion wrappers hydrate on the client.
Projects archive	Server-rendered and cached; refreshed after project publishing or reordering.
Project case study	Pre-generated or server-rendered by slug with on-demand revalidation.
Admin dashboard	Dynamic, authenticated and uncached. No admin code should enter public bundles.
API/audit routes	Server-only handlers with validation, rate limiting and permission checks.

03 / TECHNOLOGY

Recommended Technology Stack

The stack should favour first-class Next.js integration, strong TypeScript contracts, accessible primitives and predictable production deployment. Exact minor versions should be selected from current stable releases at implementation time.

Layer	Recommendation	Responsibility
Framework	Next.js App Router	Public routes, admin routes, server rendering, metadata, caching and backend handlers.
Language	TypeScript (strict)	Shared contracts across UI, database, forms and services.
UI	React	Reusable public and administrative component architecture.
Styling	Tailwind CSS v4	Design tokens, responsive layout and component styling.
Accessible primitives	Radix UI / shadcn/ui	Admin controls, dialogs, menus, forms and focus management.
Forms	React Hook Form + Zod	Performant admin forms and server-safe validation.
Database	PostgreSQL	Durable structured project and audit data.
ORM	Prisma ORM	Type-safe queries, relations and migrations.
Media	Cloudinary or S3-compatible storage	Responsive project media, transformation and CDN delivery.
Authentication	Auth.js or Clerk	Private admin access and secure session management.
Hosting	Vercel	Next.js deployment, previews, edge delivery and analytics integration.

Motion stack

Tool	Use it for	Do not use it for
Motion for React	Entrances, hover states, layout transitions, reveal choreography.	Complex pinned scroll timelines.
GSAP + ScrollTrigger	Advanced scroll-linked depth, pinned sequences and coordinated timelines.	Every button or simple fade.
Lenis (optional)	Controlled smooth-scroll feel and scroll normalisation.	Masking poor layout or slowing touch scrolling.
CSS transforms	Ambient drift, subtle looping effects and lightweight interactions.	State-heavy sequences needing orchestration.
Intersection Observer	Starting non-complex reveals only when needed.	Continuous scroll calculations.

MOTION RULE

Use the lightest tool capable of the effect. No library should control every animated element.

Testing and observability

Concern	Tools
Unit and domain logic	Vitest
Component behaviour	React Testing Library
End-to-end flows	Playwright
Accessibility automation	Axe through Playwright
Performance regression	Lighthouse CI
Errors and traces	Sentry
Product analytics	Vercel Web Analytics and/or Google Analytics
Search monitoring	Google Search Console

04 / INFORMATION ARCHITECTURE

Routes and Project Content Model

```
/
/projects
/projects/[slug]
/about              (optional dedicated page)
/contact            (optional dedicated page)
/resume
/privacy
/admin/login
/admin/dashboard
/admin/projects
/admin/projects/new
/admin/projects/[id]/edit
/admin/media
/admin/audits
/admin/testimonials
/admin/technologies
/admin/settings
```

Project case-study structure

Each project must be structured data rather than a single unrestricted rich-text field. This creates consistency, supports search and metadata, and allows the public renderer to remain performant.

Group	Recommended fields
Identity	Title, slug, short summary, project type, industries, year, status and display order.
Links	Live URL, repository URL, repository visibility and optional demo URL.
Visuals	Card image, cover image, mobile screenshots, gallery, video and architecture diagrams.
Narrative	Overview, problem, goals, role, approach, challenges, solutions, outcome and lessons.
Technology	Frontend, backend, database, deployment and third-party integrations.
Results	Business metrics, technical metrics, PageSpeed audits and measurable improvements.
Publishing	Draft/published state, featured flag, publication dates, SEO title, description and social image.

Flexible section builder

A controlled ProjectSection model should allow ordered blocks without turning the dashboard into an unsafe free-form page builder.

- Rich text
- Single image
- Image gallery
- Metrics grid
- Quote or testimonial
- Code sample
- Before-and-after comparison
- Architecture diagram
- Video or demo
- Two-column content

DESIGN CONSTRAINT

The section builder should be flexible enough for different case studies, but every block must render through a tested, accessible component.

05 / DATA MODEL

Database and Publishing Architecture

The database should preserve editorial flexibility, traceability and audit history while keeping common public queries straightforward.

```
User
Project
ProjectSection
ProjectImage
Technology
ProjectTechnology
ProjectMetric
PerformanceAudit
Testimonial
Client
MediaAsset
SiteSetting
ContactSubmission
AuditLog
```

Core relationships

```
Project
  |-- many ProjectSection
  |-- many ProjectImage
  |-- many Technology through ProjectTechnology
  |-- many ProjectMetric
  |-- many PerformanceAudit
```

Publishing lifecycle

```
Draft -> Preview -> Content validation -> SEO validation -> Publish -> Cache revalidation -> Public update
```

- **Drafts remain private:** Unpublished slugs must never appear in the sitemap or public archive.
- **Preview uses controlled access:** The administrator can inspect the exact public renderer before publishing.
- **Publishing triggers revalidation:** The home page, archive, project slug and sitemap refresh immediately.
- **Auditable changes:** Important updates and destructive actions are recorded in AuditLog.

- **Archive before deletion:** Projects should usually be archived or unpublished instead of permanently removed.

Data validation

- Unique, URL-safe slugs
- Required alt text before media can be published
- Valid public URLs for project and audit targets
- Controlled status values and section types
- Metric values stored with units and display labels
- SEO title and description length guidance
- Safe HTML or structured rich-text output only

06 / CONTENT OPERATIONS

Administration Dashboard

The dashboard should make publishing feel seamless while preventing incomplete or misleading project entries from reaching the public site.

Primary dashboard modules

Module	Capabilities
Dashboard	Project totals, drafts, recent changes, audit health and quick actions.
Projects	Create, edit, duplicate, preview, publish, archive, feature and reorder projects.
Case-study builder	Add and reorder validated content blocks.
Media library	Upload, search, reuse, replace and safely remove assets.
Technologies	Manage reusable stack labels, categories and icons.
Performance audits	Run mobile/desktop tests, review history and select the current public result.
Testimonials	Create, edit, publish and reorder social proof.
Settings	Contact details, social profiles, resume, homepage copy and global SEO defaults.
Audit log	Review important administration actions.

Project editor workflow

1. Create the project identity and draft slug.
2. Add card and hero media with required alt text.
3. Build the narrative using ordered section blocks.
4. Select technologies, roles, industry and project type.
5. Add outcome metrics and external links.
6. Configure metadata and social image.
7. Preview the actual public case-study route.
8. Run validation and publish.
9. Trigger public cache revalidation.

Security requirements

- No public registration flow
- HTTP-only secure sessions
- Protected admin routes and server-side permission checks
- Login rate limiting and optional two-factor authentication
- Strict file type and size validation
- CSRF-safe form and mutation strategy

- Safe URL validation for audit requests
- Audit logging for publishing and deletion
- Secret management through deployment environment variables

07 / VERIFIED RESULTS

Google Performance Audit Integration

Project performance results should be fetched from Google services and stored with provenance. They must never be presented as timeless or manually invented figures.

Data to capture

Category	Stored values
Scores	Performance, accessibility, best practices and SEO for mobile and desktop.
Core metrics	LCP, CLS, FCP, Speed Index, Total Blocking Time and INP where returned.
Provenance	Tested URL, audit date, strategy, Lighthouse version and result source.
History	Every successful run retained rather than overwritten.
Raw details	Selected recommendations or a normalised snapshot for future inspection.

Audit workflow

```
Admin enters verified project URL
  -> Server validates destination and permissions
  -> Mobile audit runs
  -> Desktop audit runs
  -> Results are normalised
  -> New historical records are saved
  -> Latest successful audits become current
  -> Public case study updates
```

Lab data versus field data

Data type	Meaning	Display rule
Lighthouse / PageSpeed lab data	Synthetic controlled test useful for repeatable technical comparisons.	Label as lab data and always show the audit date.
Chrome UX Report field data	Aggregated real-user Chrome experience; may be unavailable for low-traffic sites.	Show only when sufficient data exists and label as real-user data.

INTEGRITY RULE
A failed audit must never replace the latest successful result. Public cards must show the date and device strategy.

Operational safeguards

- Rate-limit audit execution
- Queue or serialize repeated audit jobs
- Validate destinations to reduce server-side request abuse
- Store failures separately from successful results
- Allow manual retry without duplicating active jobs
- Avoid running audits during every public request

08 / EXPERIENCE DESIGN

Premium Motion and Parallax System

The motion language should create believable depth, not decorative noise. Elements move at different rates because they occupy different visual layers. The system should be subtle during reading and cinematic only at selected transition points.

Three motion levels

Level	Purpose	Examples
Ambient	Barely noticed depth that keeps the page alive.	Grain, drifting grid, slow gradient, tiny background displacement.
Interaction	Immediate response to intentional user input.	Magnetic buttons, shallow card tilt, image depth, link underline and cursor labels.
Cinematic	Rare, memorable moments that shape the journey.	Loader reveal, hero entrance, selected-work transitions and blue testimonial environment.

Custom loader sequence

10. Begin on a restrained white or near-white canvas with subtle texture.
11. Reveal the Code with Sleek monogram or code symbol at the centre.
12. Construct thin interface lines outward to suggest a system being assembled.
13. Mask-reveal the phrase “Crafting logic, the sleek way.” in depth layers.
14. Move background, mark and typography at slightly different rates.
15. Split, peel or travel through the loader plane to expose the hero underneath.
16. Allow the hero to settle into its ambient parallax state.

Recommended first-session duration: approximately 1.8 to 2.6 seconds. Repeat visits in the same session should receive a much shorter transition. The loader must never wait for every image or trap the user.

Section choreography

Section	Motion direction
Hero	Layered heading, slow background grid, subtle pointer spotlight and slower heading scroll rate.
Selected Works	Multi-plane project imagery, 1-3 degree hover tilt, light-follow

	effect and restrained scroll offsets.
Process	Growing timeline, alternating depth entries and staggered step content.
Build with Sleek	Gradual environment transition into blue, floating logos and testimonial card depth.
About	Portrait and name layers moving independently with small metadata labels.
Page transitions	Short masked or depth transitions; content remains accessible throughout.

Mobile motion profile

- Reduced displacement distances
- No cursor effects or hover-only information
- Fewer simultaneous layers
- Minimal blur and no expensive pinned sequences by default
- Touch-native card interactions
- Shortened loader
- Immediate reduced-motion fallback

EXPERIENCE TARGET

Minimal when stationary. Layered while scrolling. Responsive when touched. Cinematic only when it matters.

09 / QUALITY STRATEGY

Performance, SEO and Accessibility

Performance architecture

- **Server-first public pages:** Only interactive motion islands become Client Components.
- **Transform-led animation:** Prefer transform and opacity; avoid continuous layout and expensive paint work.
- **Deferred motion:** Below-fold animation modules initialize only when needed.
- **Responsive media:** Explicit dimensions, correct sizes, modern formats and mobile crops.
- **Disciplined fonts:** At most one variable display family and one variable body family.
- **Third-party restraint:** Every external script must justify its performance cost.
- **Dedicated mobile profile:** Desktop choreography is not simply squeezed onto mobile.

Release score targets

Lighthouse category	Hard minimum	Preferred target
Performance - mobile	90	95+
Accessibility	95	100
Best Practices	95	100
SEO	95	100

Core Web Vitals engineering targets

Metric	Target
Largest Contentful Paint (LCP)	2.5 seconds or faster
Interaction to Next Paint (INP)	200 ms or faster
Cumulative Layout Shift (CLS)	0.10 or lower

Lighthouse scores vary with simulated CPU, network conditions, content and third-party services. The practical guarantee is a release process that blocks regressions below the agreed threshold, not a claim that every future run will return an identical number.

SEO requirements

- Unique title and description for every project
- Canonical URLs and indexing controls
- Dynamic Open Graph and social images
- XML sitemap containing published content only
- Robots configuration and Search Console verification

- Breadcrumbs and meaningful internal linking
- Person, CreativeWork or SoftwareApplication structured data where appropriate
- Semantic heading hierarchy and descriptive image alt text
- Valid publication and modification dates
- Custom not-found and redirect handling

Accessibility requirements

- Complete keyboard operation
- Visible focus states
- Correct landmarks and heading order
- Sufficient colour contrast
- No information available only on hover
- Reduced-motion support
- Accessible form labels and errors
- Dialog focus trapping and restoration
- Meaningful image alternatives
- Automated Axe checks plus manual screen-reader smoke tests

10 / ENGINEERING FOUNDATION

Folder Structure and Standards

```
codewithsleek-portfolio/
|-- prisma/
|   |-- schema.prisma
|   |-- migrations/
|   `-- seed.ts
|-- public/
|-- src/
|   |-- app/
|   |   |-- (website)/
|   |   |-- admin/
|   |   |-- api/
|   |   |-- sitemap.ts
|   |   `-- robots.ts
|   |-- components/
|   |   |-- ui/
|   |   |-- layout/
|   |   |-- admin/
|   |   `-- shared/
|   |-- features/
|   |   |-- home/
|   |   |-- projects/
|   |   |-- performance-audits/
|   |   |-- media/
|   |   `-- authentication/
|   |-- motion/
|   |   |-- components/
|   |   |-- hooks/
|   |   |-- timelines/
|   |   `-- parallax/
|   |-- server/
|   |   |-- actions/
|   |   |-- queries/
|   |   |-- services/
|   |   `-- permissions/
|   |-- lib/
|   |-- styles/
|   |-- types/
|   `-- config/
|-- tests/
|   |-- unit/
|   |-- integration/
|   |-- e2e/
|   |-- accessibility/
|   `-- lighthouse/
|-- .github/workflows/
```

```
|-- lighthouserc.*
|-- next.config.*
`-- package.json
```

Engineering conventions

- Strict TypeScript and validated environment variables
- Feature-oriented modules rather than one giant components folder
- Database access isolated to server code
- Public components must not import admin dependencies
- Schema validation at every external boundary
- No project-specific hard-coded page JSX
- Motion presets centralised in a motion configuration
- Design tokens defined once and consumed consistently
- All important mutations covered by permissions and audit logging
- Lint, typecheck, tests and production build required in CI

Testing and Release Gates

Automated test layers

Layer	Coverage
Unit	Slug logic, score normalisation, metadata helpers, permission rules and content transformations.
Integration	Database operations, publishing lifecycle, audit storage and cache invalidation.
Component	Forms, section renderers, dialogs, validation and accessible interactions.
End-to-end	Administrator publishing and the complete public browsing journey.
Accessibility	Automated Axe checks and keyboard-focused scenarios.
Visual regression	Critical responsive layouts and animation-disabled snapshots.
Lighthouse CI	Performance, accessibility, best practices and SEO thresholds on production-like previews.
Security smoke tests	Authentication boundaries, unsafe URL rejection and protected mutations.

Critical end-to-end flow

17. Administrator signs in.
18. Creates a project.
19. Uploads and assigns media.
20. Adds structured case-study sections.
21. Previews the project.
22. Publishes the project.
23. The project appears on the landing page and archive.
24. The detail route renders correctly.
25. Metadata and social preview are correct.
26. A performance audit runs and its dated results display.

Continuous integration gate

```
install -> lint -> typecheck -> unit/integration tests -> build -> E2E -> accessibility -> Lighthouse CI -> deploy approval
```

MERGE POLICY
A pull request cannot merge when critical tests fail or the agreed mobile Lighthouse floor is breached.

Sprint Execution Plan

The plan is divided into twelve delivery sprints plus a discovery sprint. Sprint length can be one or two weeks; the sequence should remain largely unchanged because each stage prepares the quality conditions for the next.

Sprint 0 - Product definition and technical specification

Deliverables	Final route inventory, content model, admin scope, motion map, browser policy, performance budgets, backlog and definition of done.
Exit approval	Every route and V1 feature is agreed; no unresolved decision blocks implementation.

Sprint 1 - Repository and engineering foundation

Deliverables	Next.js setup, strict TypeScript, Tailwind, linting, formatting, environment validation, CI, route groups and initial structure.
Exit approval	Lint, typecheck, tests and production build pass.

Sprint 2 - Design system and static shell

Deliverables	Typography, colours, spacing, containers, responsive breakpoints, buttons, navigation, footer and focus states.
Exit approval	Desktop and mobile foundations are visually approved and keyboard usable.

Sprint 3 - Database, authentication and admin shell

Deliverables	PostgreSQL, Prisma schema, migrations, seed, secure login, protected routes, permissions and

	dashboard layout.
Exit approval	Unauthenticated access is blocked and the database can be recreated safely.

Sprint 4 - Project content management

Deliverables	Project CRUD, drafts, slugs, section builder, technologies, reordering, preview and publishing validation.
Exit approval	A project can be created and published without editing code.

Sprint 5 - Media pipeline

Deliverables	Secure uploads, media validation, alt text, reusable library, card/cover/gallery assignment and responsive delivery.
Exit approval	Invalid media is rejected and published media has dimensions and alt text.

Sprint 6 - Public landing page

Deliverables	Hero, metrics, selected work, process, testimonials, about, contact and footer using real data.
Exit approval	Responsive page is approved before advanced motion begins.

Sprint 7 - Project archive and case studies

Deliverables	Archive, dynamic slug pages, section renderer, gallery, results, navigation, related projects, metadata and error states.
Exit approval	Three structurally different projects render without project-specific JSX.

Sprint 8 - Premium loader and motion

Deliverables	First-session loader, hero parallax, project depth, process choreography, blue-section transition, about motion and reduced-motion/mobile profiles.
Exit approval	No inaccessible content, no overflow and no representative mobile jank.

Sprint 9 - Google audit integration

Deliverables	PageSpeed service, mobile/desktop runs, normalisation, history, retry handling, admin UI and public display.
Exit approval	Audits are traceable; failures never replace successful results.

Sprint 10 - SEO, accessibility and content hardening

Deliverables	Metadata, sitemap, robots, structured data, social cards, heading/alt audits, keyboard and contrast review.
Exit approval	No critical Axe violations and all indexable routes have unique metadata.

Sprint 11 - Automated quality gates

Deliverables	Unit, integration, E2E, accessibility, visual regression, Lighthouse CI, broken-link and bundle checks.
Exit approval	All critical CI gates pass and regressions block merging.

Sprint 12 - Production release and approval

Deliverables	Production services, domain, security headers, monitoring, analytics, Search Console, backup,

	rollback and device testing.
Exit approval	Version 1.0 passes the final release checklist.

13 / RELEASE

Version 1.0 Approval Checklist

- [ ] Mobile Lighthouse performance is 90 or higher on agreed production-like routes.
- [ ] Accessibility, best practices and SEO are each 95 or higher.
- [ ] No critical automated accessibility violations remain.
- [ ] All critical end-to-end tests pass.
- [ ] No public links are broken.
- [ ] All published projects have valid metadata and social previews.
- [ ] Admin project publishing works from login through public display.
- [ ] PageSpeed results show source, date and mobile/desktop strategy.
- [ ] Mobile parallax remains smooth on representative devices.
- [ ] Reduced-motion mode provides an immediate, usable experience.
- [ ] Error monitoring and analytics are active.
- [ ] Backups and rollback procedures are documented.
- [ ] Search Console and sitemap submission are complete.
- [ ] No unpublished project appears publicly or in the sitemap.

Final device matrix

Environment	Required verification
iPhone Safari	Layout, loader, scrolling, parallax, navigation and contact actions.
Android Chrome	Performance, touch interactions, images and case-study reading.
Desktop Chrome	Full cinematic experience, keyboard and Lighthouse baseline.
Desktop Safari	Animation compatibility, fonts, media and sticky behaviour.
Firefox	Layout, forms, focus states and content fallbacks.
Slow mobile simulation	LCP, layout stability, deferred motion and media loading.
Reduced motion	No scroll-linked displacement; content immediately usable.
Keyboard and screen reader	Navigation, dialogs, forms, landmarks and reading order.

Future Roadmap, Risks and Decisions

Recommended post-V1 roadmap

- Publish an Insights section for technical writing and case-study lessons
- Add project view and CTA conversion analytics
- Support scheduled publishing and richer editorial previews
- Introduce multiple admin roles only when another contributor exists
- Add automated Core Web Vitals monitoring and regression alerts
- Create reusable project comparison and technology archive pages
- Add multilingual content only if audience data supports it

Key risks and mitigations

Risk	Mitigation
Motion damages mobile performance	Mobile-specific motion profile, transform-only rules, deferred modules and Lighthouse gates.
Overbuilding the administration system	Ship controlled project blocks first; defer a general-purpose page builder.
Inconsistent project case studies	Required fields, content templates, previews and publishing validation.
Misleading performance claims	Store dated audit provenance and clearly label lab versus field results.
Large media causes poor LCP	Responsive CDN transformations, dimensions, compression and strict upload rules.
Third-party scripts reduce scores	Maintain an explicit script budget and load non-critical tools after interaction or consent.
SEO score confused with ranking	Treat Lighthouse SEO as technical hygiene; continue content and authority work separately.
Animation maintenance becomes fragile	Central motion presets, clear ownership between Motion and GSAP, reduced-motion tests.

Architecture decisions to confirm before Sprint 1

27. Cloudinary versus S3-compatible object storage.
28. Auth.js versus Clerk for the single-admin authentication flow.
29. Whether the first release needs a dedicated /about and /contact route.
30. Whether project source repositories can be private and hidden.
31. Which PageSpeed metrics appear publicly on cards versus only inside case studies.
32. Whether Lenis improves the final mobile experience enough to justify its inclusion.
33. The final display and body font families from the design system.

FINAL RECOMMENDATION

Build a server-first Next.js portfolio platform with PostgreSQL, Prisma, secure administration, structured case studies, verified PageSpeed records and a carefully budgeted Motion + GSAP experience.

CODE WITH SLEEK

Crafting logic, the sleek way.